

SDK Docs Template

开发文档

从结构化 Markdown 生成双语文档站、多版本入口与严格排版 PDF。

文档版本 **3.0.0**

更新时间 2026-10-10

目录

项目概览

- 1. 能力地图与验证边界 1
 - 1.1 输出体验 1

快速开始

- 1. 首次本地构建 2
 - 1.1 预期产物 2
 - 1.2 首次失败时先做什么 2

内容创作

- 1. 内容约定：让文档可以长期维护 3
 - 1.1 文件命名 3
 - 1.2 每篇技术页面的最小结构 3
- 2. 双语编写与语言切换 4
 - 2.1 对应页规则 4
 - 2.2 译文质量门槛 4

内容组织

- 1. 选择递归文档树还是严格项目目录 5
 - 1.1 选择 `recursive_tree` 5
 - 1.2 选择 `project_catalog` 5

构建与输出

- 1. 本地构建工作流 6
- 2. PDF 交付：排版与环境同样是构建输入 7
 - 2.1 输出特征 7

版本与部署

- 1. GitHub Actions 与 GitHub Pages 自动发布 8
 - 1.1 工作流阶段 8
 - 1.2 PR 与默认分支的行为差异 8
 - 1.3 发布前检查清单 8

场景化示例

- 1. 示例：嵌入式 SDK 开发手册 9
 - 1.1 场景 9
 - 1.2 内容策略 9
 - 1.3 验收 9
- 2. 示例：BSP 与驱动项目目录 10
 - 2.1 场景 10
 - 2.2 为什么这比递归扫描更可靠 10
- 3. 示例：可归档的 PDF 发布手册 11
 - 3.1 场景 11
 - 3.2 建议的内容压力测试 11
- 4. 示例：多版本 GitHub Pages 发布 12
 - 4.1 场景 12

配置参考

- 1. 高频配置 13
- 2. 常用命令 13
- 3. 页面相关实现 14

问题排查

- 1. 排查顺序 15

维护者指南

- 1. 架构职责 16
- 2. Web 导航实现边界 16
- 3. 贡献要求 16

项目概览

1. 能力地图与验证边界

模板的关键不是功能数量，而是每项功能都有明确输入、输出和失败边界。

链路阶段	输入	输出	发布前阻断项
内容发现	projects 中的文档和资源	经验证的 DocumentCatalog	越界路径、缺失本地图片、项目规则冲突
HTML 构建	内容清单和 Sphinx 配置	中文、英文独立站点	语言首页或导航生成失败
PDF 构建	同一内容清单与 LaTeX 配置	一份或两份可下载 PDF	XeLaTeX、字体、PDF 有效性校验失败
多版本构建	versions.json 与 Git 分支	隔离版本目录	分支或版本定义无效
GitHub 发布	通过验证的构建产物	Artifact 与 GitHub Pages	PR 校验未通过时不部署

1.1 输出体验

Web 端提供全文搜索、深色模式、语言切换、版本菜单、编辑入口、PDF 下载和可选的 giscus 讨论。PDF 端保留封面、目录、页眉页码、书签、章节分页、代码块、表格、图片和 WebP 转换策略。

使用模板时的判断原则

如果某项约束只在部署后才会暴露，它就应该被前移到本地检查或 CI。模板中的严格校验不是额外负担，而是为了让文档在用户看到之前就具备可交付性。

下一步可在 内容组织 选择你的仓库模型。

快速开始

1. 首次本地构建

在 source 目录执行以下命令。脚本会读取 config.yaml、检查锁定的 Python 依赖、同步 projects 内容、生成导航并分别构建语言站点。

```
python build_local.py --check
python build_local.py --clean --no-pdf
python build_local.py --clean --no-pdf --serve --port 8000
```

1.1 预期产物

路径	用途
source/_build/html/index.html	本地稳定入口，会跳转到默认语言首页
source/_build/html/	HTML、搜索索引和静态资源
source/_build/html/_static/	下载 PDF 和运行时页面资源
source/_build/latex/	PDF 中间产物，成功后会清理

若依赖缺失，脚本默认尝试安装可用版本；在受控环境中加上 --no-auto-install，使其只报告差异。执行 --clean 会清理旧 HTML，避免已删除页面仍残留在搜索或导航中。

1.2 首次失败时先做什么

先运行 python build_local.py --check --no-auto-install，确认 Python 环境与 requirements.txt 相符。若 HTML 可以生成但 PDF 失败，不要尝试用本机替代字体绕过错误；请转到[PDF 交付](#)按名称安装配置要求的字体。

内容创作

1. 内容约定：让文档可以长期维护

1.1 文件命名

内容类型	中文	英文
网站或章节首页	README_zh.md	README.md
正文 Markdown	topic_zh.md	topic.md
正文 reStructuredText	topic_zh.rst	topic.rst
图片和附件	figures/ 或 assets/	同目录共享

同一篇双语内容必须放在同一目录并使用同一基础名，例如 `install_zh.md` 对应 `install.md`。目录和文件名使用稳定、可读的英文 `snake_case`；展示给读者的中文标题写在文档一级标题和 `config.yaml` 的分类名称中。

1.2 每篇技术页面的最小结构

1. 先说明目标和适用版本。
2. 列出不可省略的前置条件。
3. 给出可以直接运行的步骤，并标明运行环境。
4. 提供成功判据，例如日志、文件、界面状态或命令输出。
5. 写出常见失败的下一跳链接。

图片必须跟随文章存放并使用相对路径。构建器会校验本地图片是否真实存在且位于 `projects` 范围内，因此不要使用看似可用的 `../` 相对逃逸路径。

专业文档的表达原则

命令不是结论。每段命令都应说明执行位置、意图、预期变化和回滚或排查入口。读者需要的是可验证的行动，不是一串无法判断成功与否的指令。

2. 双语编写与语言切换

双语不是把同一页面混合两种语言。模板分别构建中文和英文源文件，各自拥有导航、搜索索引和静态资源。只有检测到两种语言的根 README 时，页面才显示切换控件。

2.1 对应页规则

当前页面	目标语言页面存在	目标页面不存在
setup_zh.md	跳转 setup.md	跳转英文首页
setup.md	跳转 setup_zh.md	跳转中文首页

这项回退策略允许团队渐进翻译，不会因为某一篇英文尚未完成而导致切换按钮失效。完成翻译时只需补齐同基础名的文件，不需要修改前端逻辑。

2.2 译文质量门槛

接口名、命令、文件路径、配置字段和值保持原样；解释性文字按目标读者重写，不逐词翻译。发布前在两种语言下分别搜索关键术语，并从多个深层页面测试语言切换，确保链接不是只在首页正确。

内容组织

1. 选择递归文档树还是严格项目目录

1.1 选择 `recursive_tree`

当文档目录本身就是读者的学习路线时，使用 `recursive_tree`。它递归同步 Markdown、RST 与图片，并使用目录 `README` 作为章节首页。适用于安装指南、API 手册、产品教程和架构知识库。

```
projects/
├── getting_started/
│   ├── README_zh.md
│   └── install_zh.md
└── reference/
    ├── README_zh.md
    └── configuration_zh.md
```

1.2 选择 `project_catalog`

当仓库同时含有源码、供应商包、构建输出和多个独立示例时，使用 `project_catalog`。通过 `categories.*.patterns` 选择项目根目录，仅收录 `entry_files` 和 `asset_globs` 声明的文件。

```
projects/
├── board_alpha/
│   ├── README_zh.md
│   ├── README.md
│   └── figures/
└── vendor/
    └── third_party_library/
```

严格模式会拒绝空匹配、重复分类、缺少项目入口 `README`、越界路径和未同步图片。它的价值在于明确地证明“发布了什么”和“没有发布什么”。

不要混合职责

不要用目录命名技巧模拟项目白名单，也不要使用分类模式代替一本连续手册的章节层级。先选择发现模型，再配置导航，文档结构会更易维护。

构建与输出

1. 本地构建工作流

`build_local.py` 是单个版本的本地入口。它支持清理、依赖检查、静态服务、端口设置、跳过 PDF、禁止自动安装和分支检查等常用工作流。

参数	作用
<code>--clean</code>	构建前清理旧 HTML 输出
<code>--serve</code>	构建后启动静态服务器
<code>--port 8000</code>	指定静态服务器端口
<code>--no-pdf</code>	跳过 XeLaTeX，专注网页反馈
<code>--check</code>	检查依赖和构建环境
<code>--no-auto-install</code>	只检查，不安装 Python 或 PDF 依赖
<code>--check-branch</code>	验证当前分支是否符合版本配置

构建完成后，检查首页跳转、左右侧栏、搜索、深层链接、语言切换和静态图片。不要只看首页：目录首页、第三层页面和不存在翻译的回退页面最容易暴露路径问题。

桌面端还应确认页面按左侧全局目录、中间正文、右侧文章大纲排列；调整窗口宽度时正文不突然跳宽，空间不足时大纲才移动到正文下方。文章大纲中的 `h2/h3` 应保持嵌套缩进，点击标题后平滑滚动并更新高亮；在左侧切换文章后，之前展开的目录路径应保持。

使用 `--serve` 时脚本以前台运行；结束服务可在终端按 `Ctrl+C`。若端口已被占用，请使用 `--port` 指定另一个未使用端口。

2. PDF 交付：排版与环境同样是构建输入

PDF 不是从浏览器打印网页。模板固定使用 Sphinx LaTeX 到 XeLaTeX 的路径，并在开始排版前验证工具和精确字体。中文和英文按实际检测到的语言分别生成 PDF。

```
DocumentCatalog
    ↓
Sphinx LaTeX builder
    ↓
XeLaTeX + configured fonts
    ↓
validated Chinese / English PDF
```

2.1 输出特征

封面、目录、页眉、页码、书签和章节分页由 LaTeX 配置统一控制。表格、代码块、行内代码、图片和 WebP 均有专门处理；PDF_STYLE 可在 web、thesis、graduate 和 academic 之间选择。

```
python utils/pdf_environment.py --no-auto-install
python build_local.py --clean
```

生成文件位于 source/_build/html/_static，中文文件名通常是 SDK_Docs_Template.pdf，英文文件名以 _EN.pdf 结尾。

严格字体校验是有意设计

字体缺失时构建会失败，不会静默回退到相近字体。这样才能避免本地正常、CI 乱码，或不同机器的分页和代码宽度不一致。

版本与部署

1. GitHub Actions 与 GitHub Pages 自动发布

仓库内置 workflow 监听 source、projects 和 versions.json 的变化。它把“配置可解析”“PDF 环境可用”“所有版本可以构建”“产物能部署”拆成可观察的阶段，而不是在最后一步才发现问题。

1.1 工作流阶段

Job 或步骤	责任	结果
validate-versions	安装 Python 3.11 与依赖，运行 <code>build.py --validate</code>	阻止无效版本配置进入构建
build-docs	安装 XeLaTeX、字体和 locale，验证 PDF 环境	为每个版本生成 HTML 和 PDF
upload-artifact	上传 source/source_build/html	Pull Request 上可下载并人工验收
deploy-docs	仅在 main 或 master 下载 Artifact 并发布	GitHub Pages 更新到孤儿分支

1.2 PR 与默认分支的行为差异

Pull Request 会运行校验和构建，但不会部署；这给评审者一个可下载 Artifact，用来检查导航、语言切换、搜索和 PDF。合并到 main 或 master 后，deploy-docs 通过 peaceiris/actions-gh-pages 发布同一份 Artifact，并以 force_orphan 保持发布分支只保存静态站点。

1.3 发布前检查清单

- versions.json 已通过 `build.py --validate`。
- 无 PDF 的本地构建已通过，且深层页面和图片可访问。
- 在具备 XeLaTeX 与精确字体的环境中已完成完整 PDF 构建。
- 英文和中文页面都检查过搜索与切换回退。
- Artifact 内容与 Pages 入口版本一致。

触发条件和环境安装命令在 `.github/workflows/build-docs.yml` 中；不建议把关键依赖只安装在本地而遗漏 CI。

场景化示例

1. 示例：嵌入式 SDK 开发手册

1.1 场景

一个传感器 SDK 同时需要安装、初始化、数据采集、错误码和板级排错文档。读者会沿着学习路径阅读，因此选择 `recursive_tree`，让目录结构直接成为导航结构。

```
projects/
├── getting_started/
│   ├── README_zh.md
│   ├── install_zh.md
│   └── first_sample_zh.md
├── api_reference/
│   ├── README_zh.md
│   └── sensor_stream_zh.md
└── troubleshooting/
    ├── README_zh.md
    └── i2c_diagnostics_zh.md
```

1.2 内容策略

- 安装页面给出操作系统、工具链和最低 SDK 版本。
- API 页面使用表格描述参数，使用可复制的代码块描述最小调用。
- 排错页面按症状、证据、原因和修复顺序编写，而不是按模块名堆叠。
- 同名中英文文件让语言切换保留阅读上下文。

1.3 验收

运行 `python build_local.py --clean --no-pdf --serve` 后，检查从安装页到 API 页的侧栏连续性、代码块复制体验、图片分辨率和中英文对应页。发布前再运行完整 PDF，确认长表格和代码没有越界。

2. 示例：BSP 与驱动项目目录

2.1 场景

一个 BSP 仓库包含评估板、传感器驱动、参考工程、供应商代码和构建缓存。公开站点应展示经过维护的项目入口，而不是递归扫描所有 README，因此选择 `project_catalog`。

```
categories:
  boards:
    name: "开发板"
    patterns: ["board_*"]
  drivers:
    name: "驱动"
    patterns: ["driver_*"]
generation:
  discovery:
    mode: project_catalog
    entry_files:
      zh: README_zh.md
      en: README.md
    asset_globs: ["figures/**", "assets/**"]
    unmatched_projects: error
    duplicate_categories: error
  navigation:
    mode: categories
    order: ["boards", "drivers"]
```

2.2 为什么这比递归扫描更可靠

供应商目录、包管理缓存和测试夹具经常包含 README，但它们并不属于公开支持范围。严格项目目录只发布分类规则选中的项目入口和图像资源，并阻止一个项目同时归属多个分类。

构建失败本身就是治理信号：新增加的板卡如果没有对应分类、项目没有入口 README，或 README 引用了未收录图片，CI 会直接指出问题。

3. 示例：可归档的 PDF 发布手册

3.1 场景

现场交付、客户验收和离线培训都需要一个不依赖网页的稳定文档。此时 PDF 的分页、字体、书签和代码可读性属于交付要求，不是视觉装饰。

3.2 建议的内容压力测试

内容类型	应验证的现象
长标题与中英文混排	章节层级、书签和页眉不乱码
宽表格	不超出页面，换页后表头仍清晰
代码块与行内代码	等宽字体正确，长命令不遮挡正文
PNG、JPG、WebP	方向、尺寸和清晰度稳定
多层目录	目录、书签和章节分页可跳转

通过 `PDF_STYLE=web` 默认生成产品文档；需要论文或归档预览时，可设置 `PDF_STYLE=thesis`、`graduate` 或 `academic`。无论哪种样式，都应在 CI 中安装 `config.yaml` 指定的同名字体，确保本地和云端排版一致。

最终 PDF 不是网页截图，而是经过有效性校验的文件。构建脚本会检查文件头和最小大小，下载入口根据实际生成的中文或英文文件自动呈现。

4. 示例：多版本 GitHub Pages 发布

4.1 场景

开源 SDK 的 main 分支持续迭代，但用户仍需要 v1.1 的安装命令和 API 说明。将 latest 和稳定分支同时发布，既保证新读者看到当前能力，也避免历史使用者被不兼容修改打断。

```
{
  "versions": [
    {
      "name": "main",
      "display_name": "latest",
      "branch": "main",
      "url_path": "latest",
      "description": "最新开发版本"
    },
    {
      "name": "v1.1",
      "display_name": "v1.1",
      "branch": "v1.1",
      "url_path": "v1.1",
      "description": "稳定维护版本"
    }
  ],
  "default_version": "main",
  "latest_version": "main"
}
```

构建器不会 checkout 当前工作目录，而是为 main 和 v1.1 分别建立 worktree。PR 阶段产出 Artifact 供评审；合并到默认分支后，部署任务将相同 Artifact 发布到 GitHub Pages。版本菜单只需读取构建时注入的版本描述，无需手工维护静态链接。

配置参考

本章用于查找准确字段和命令，不重复解释工作流。新项目可从 `source/config_templates` 中选择 `recursive_tree.yaml` 或 `project_catalog.yaml`，再替换项目元数据、目录和分类。

1. 高频配置

字段	用途
<code>repository.projects_dir</code>	相对 <code>source/config.yaml</code> 的内容根目录
<code>generation.discovery.mode</code>	<code>recursive_tree</code> 或 <code>project_catalog</code>
<code>generation.navigation.order</code>	顶层分类或目录显示顺序
<code>generation.navigation.maxdepth</code>	全局目录深度；-1 表示保留完整目录树
<code>generation.navigation.titles_only</code>	全局目录是否只显示文档和目录标题
<code>generation.navigation.show_local_toc</code>	是否在文章右侧生成 h2/h3 大纲
<code>generation.default_page</code>	各语言网站首页
<code>generation.directory_index</code>	每个章节目录的首页文件
<code>generation.pdf_style</code>	<code>web</code> 、 <code>thesis</code> 、 <code>graduate</code> 或 <code>academic</code>
<code>generation.pdf_fonts</code>	本地和 CI 必须具备的精确字体

2. 常用命令

命令	目的
<code>python build_local.py --check</code>	检查本地构建环境
<code>python build_local.py --clean --no-pdf</code>	快速生成网页
<code>python build.py --validate</code>	校验 <code>versions.json</code>
<code>python build.py --list-versions</code>	列出版本配置
<code>python build.py --clean</code>	构建全部版本

配置字段的行为说明见 [内容组织](#)、[构建与输出](#)和 [版本与部署](#)。

3. 页面相关实现

文件	作用
source/_templates/layout.html	输出左侧全局树、正文容器和右侧大纲挂载点
source/_static/custom.css	控制三栏宽度、容器查询、层级缩进和窄屏堆叠
source/_static/page_outline.js	从当前正文提取 h2/h3，生成纯文本嵌套大纲并更新阅读高亮
source/_static/navigation_state.js	维护唯一展开路径和页面/锚点跳转后的导航状态

问题排查

遇到构建失败时，先根据错误所属阶段缩小范围。不要直接删除 source 中的临时文件或修改生成导航；这些文件由构建器清单管理，下一次成功构建会自动清理。

现象	首先检查	推荐命令
页面或图片缺失	projects 中的相对路径、文件后缀、目录范围	<code>python build_local.py --clean --no-pdf</code>
双语按钮不存在或跳转首页	根 README 对、同名翻译文件	<code>python build_local.py --clean --no-pdf</code>
分类为空或项目重复	patterns、entry_files、duplicate_categories	<code>python build.py --validate</code>
PDF 字体或 XeLaTeX 失败	config.yaml 字体名和 pdf_environment	<code>python utils/pdf_environment.py --no-auto-install</code>
CI 与本地不一致	Python 版本、requirements、字体、locale	对照 workflow 的安装步骤

1. 排查顺序

1. 运行 `--check --no-auto-install`，记录依赖差异。
2. 运行 `--clean --no-pdf`，先把内容与导航问题同 PDF 环境问题分开。
3. 检查生成目录中目标语言的页面和图片。
4. 再运行完整 PDF 构建，按缺失工具或精确字体修复。
5. 最后对照 GitHub Actions Artifact，而不是只看 Pages 缓存。

维护者指南

模板的维护边界是：内容作者只需要维护 `projects`，构建器维护者负责 `source` 中的发现、同步、导航、HTML、PDF、版本和测试模块。任何新功能都应明确它影响的内容清单、语言行为、HTML 输出、PDF 输出和 CI 验证。

1. 架构职责

模块	职责
DocumentCatalog	发现文档和资源，执行路径与分类约束
FileProcessor 与 IndexGenerator	同步内容，生成语言化导航和目录首页
html_builder	分别构建语言站点并写入稳定入口
pdf_builder 与 pdf_environment	验证 PDF 环境，生成并校验文件
build.py 与 build_manager	解析版本矩阵，隔离 <code>worktree</code> 并汇总产物

2. Web 导航实现边界

`layout.html` 只负责输出结构，不把文章章节混入全局 `toctree`。`custom.css` 以内容容器为基准计算正文和大纲宽度，并在空间不足时堆叠；`page_outline.js` 只消费当前正文的 `h2/h3`，使用纯文本生成嵌套层级和阅读高亮；`navigation_state.js` 只管理左侧树的唯一展开路径和跨页面恢复。修改其中任一模块时，应同时检查桌面三栏、窄屏单列、深层目录和页内锚点跳转。

3. 贡献要求

修改发现或导航逻辑时，至少覆盖 `recursive_tree`、`project_catalog`、中文、英文、双语和缺失翻译回退。修改 PDF 逻辑时，除单元测试外，应在包含 `XeLaTeX` 和配置字体的环境中验证生成文件有效。

维护者原则

不要把某个输出端的临时修复变成另一个输出端的隐患。HTML、PDF、资源和语言识别应优先复用同一份 `DocumentCatalog`，而不是各自重新扫描文件系统。